Why do we need Graphics Processing Unit (GPUs)?

Suppose you want to:

- Multiply two 10,000 × 10,000 matrices
- Train a neural network
- Perform encryption on millions of blocks

A CPU processes a few threads at a time.

A GPU processes thousands simultaneously.

Analogy:

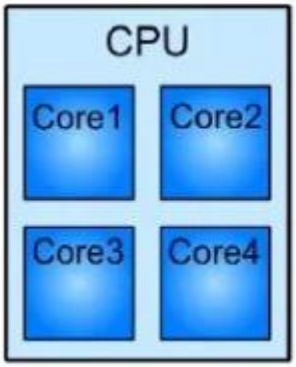
- CPU: 4 intelligent students solving 4 hard problems
  GPU: 5000 average students solving 5000 similar small problems
- For repetitive large-scale tasks → GPU wins

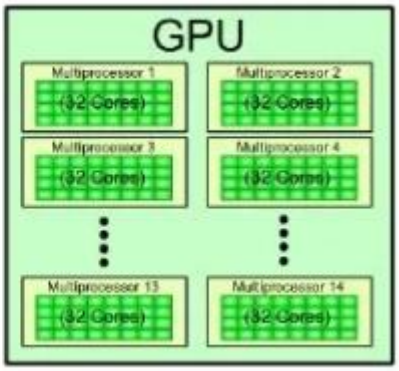
Reason:

- GPUs use SIMT (Single Instruction Multiple Threads) model.
- Each group of 32 threads : Warp
- All 32 execute the same instruction simultaneously.

CPU vs. GPU: The Fundamental Difference

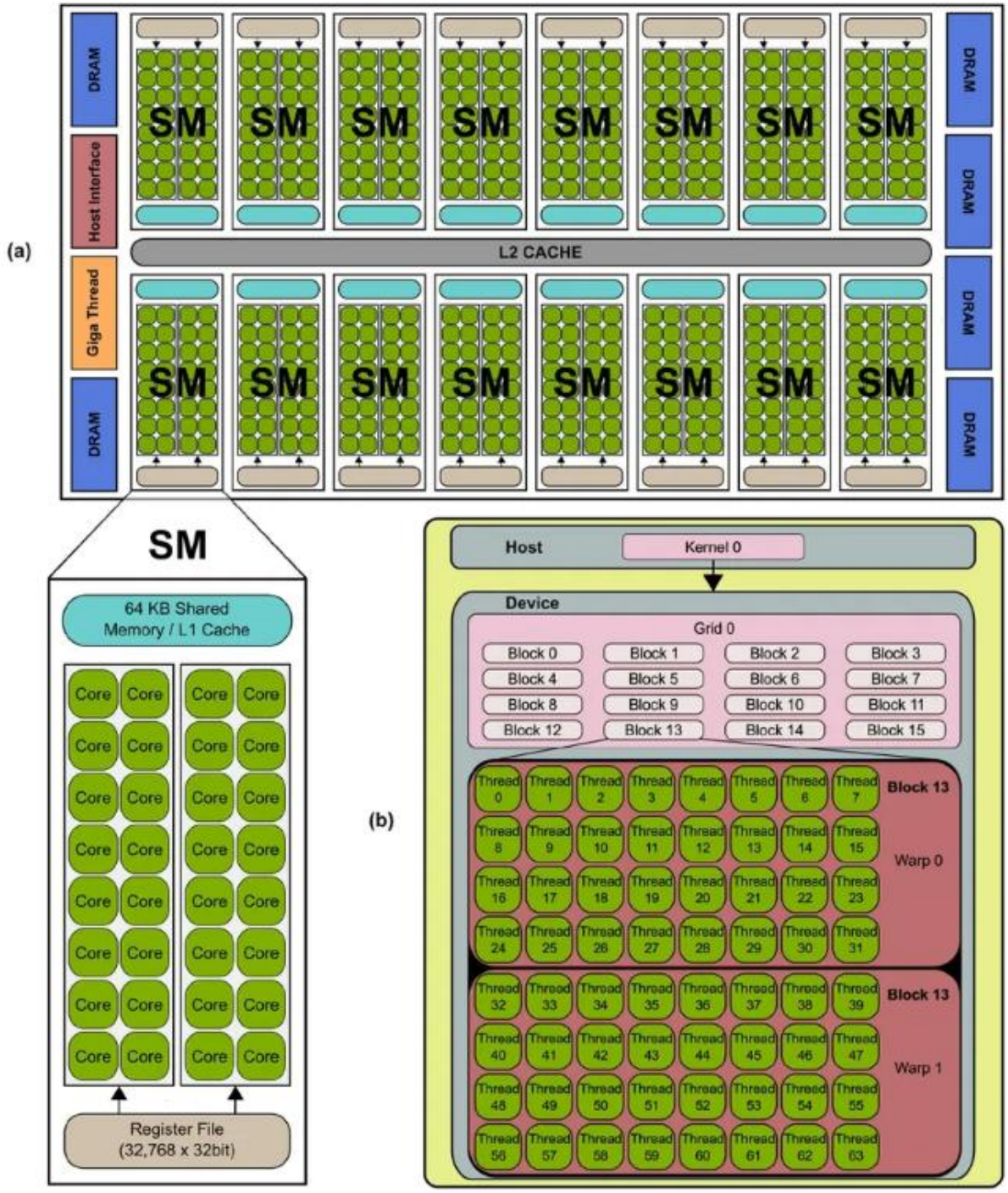
- A CPU (Central Processing Unit) and a GPU (Graphics Processing Unit) both process data, but they do so in fundamentally different ways:





- **CPU:** Has a few powerful cores that process tasks sequentially or with limited parallelism.
- **GPU:** Contains thousands of smaller cores designed for massive parallelism, making it ideal for tasks like rendering images, training AI models, and simulations.
- While a CPU is optimized for general-purpose computing, a GPU excels in handling thousands of smaller, independent computations simultaneously.

GPU Architechture:



The top part of the diagram (a) shows the entire GPU chip layout.

A GPU is composed of many parallel processing units called Streaming Multiprocessors (SMs) connected to memory and host communication units.

Host Interface:

Function: The Host Interface connects the CPU to the GPU.

It performs:

- Receiving instructions from the CPU
- Launching GPU kernels
- Transferring data between CPU and GPU

GigaThread Engine:

Function:

The GigaThread Engine is the GPU's global thread scheduler.

It performs:

- Distributes thread blocks to available SMs
- Handles thousands of threads simultaneously
- Ensures load balancing across SMs

Streaming Multiprocessor (SM): Each SM is a parallel processor containing many CUDA cores.

Function:

The SM executes:

- CUDA threads
- arithmetic operations
- memory instructions

Each SM executes multiple thread blocks simultaneously.

CUDA Cores:

Inside each SM there are many CUDA cores.

Function:

CUDA cores perform:

- Integer operations
- Floating-point operations
- Arithmetic computations

One CUDA core executes one thread instruction per cycle, but thousands of cores allow massive parallelism.

Register File: Register File (32768 × 32bit)

Function:

Registers store private variables of each thread.

Properties:

- Fastest memory

- Accessible only by that thread
- Located inside SM

Shared Memory / L1 Cache:

Inside each SM: 64 KB Shared Memory / L1 Cache

Function

This memory is:

- Shared by threads in the same block
- Very fast (on-chip memory)

Used for:

- thread communication
- intermediate results
- reducing global memory access, because it is 100 times faster than global memory.

L2 Cache: Located at the center of the GPU chip.

Function

L2 cache is shared by all SMs.

It acts as a buffer between SMs and global memory.

Multiple SMs accessing same data can retrieve it from L2 cache instead of DRAM.

Global Memory (DRAM): This is the main GPU memory.

Properties:

Feature	Description
Size	Several GB
Speed	Slower than shared memory
Scope	Accessible by all threads

CUDA Execution Model: The bottom-right diagram (b) shows the CUDA thread hierarchy.

CUDA organizes parallel computation in five levels.

1. **Host:** CPU program that launches GPU computation.

2. **Kernel:** A kernel is a GPU function executed by many threads.

3. **Grid:** A grid contains multiple blocks. Each block executes independently.

4. **Block:** A block contains many threads. Threads in a block can synchronize and share shared memory.

5. **Warp:** A warp is a group of 32 threads executed together.
   Properties:

- Warp executes same instruction simultaneously
- Uses SIMT (Single Instruction Multiple Thread) model.

Warp Divergence: If a workload does not properly align with the warp size (isn't a multiple of 32 or 64), some cores will sit idle, reducing performance. This is called warp divergence, which can negatively impact efficiency.

```
if (threadIdx.x % 2 == 0)
  do_A();
else
  do_B();
```

Warp must execute A first, then B.

This slows down performance. This is warp divergence.

Execution Flow:

Step 1: CPU launches kernel.

Step 2: Kernel enters GigaThread Engine.

Step 3: Thread blocks distributed to SMs.

Step 4: Each SM executes blocks using warps of 32 threads.

Step 5: Threads access registers, shared memory, L1/L2 Cache, DRAM.

Step 6: Results written back to global memory.

How to Optimize GPU Performance?

1. **Reduce Memory Transfers**: Minimize the number of times data moves between DRAM and Shared Memory.
2. **Maximize Parallelism**: Ensure more programs run per SM by keeping memory usage per program low.
3. **Optimize Workloads for Warps**: Use multiples of 32 (or 64) in workloads to fully utilize all cores within a warp.

Real GPU Example: NVIDIA GeForce RTX 3060

- 28 SMs
- 3584 CUDA cores
- Warp size = 32

Occupancy: Active warps per SM / Maximum possible warps per SM

- Higher occupancy → better hardware utilization.
- But: High occupancy ≠ always high performance.
- Memory access pattern also matters.

Can two blocks share shared memory?

No, two CUDA thread blocks cannot share shared memory because shared memory is allocated per block within a Streaming Multiprocessor. Blocks may execute on different SMs or at different times, so shared memory is isolated to ensure block independence.

Why does CUDA require an NVIDIA GPU?

CUDA (**Compute Unified Device Architecture**) is a parallel computing platform and programming model developed by NVIDIA. It is designed specifically to run on NVIDIA GPUs, which contain hardware components built to support CUDA instructions. Because of this tight integration between the software platform and the GPU hardware architecture, CUDA programs require an NVIDIA GPU to execute.

Heterogeneous Computing Model:

CUDA uses two processors:

Component	Role
CPU	Host
GPU	Device

CPU:

- Controls execution
- Allocates memory
- Launches kernel

GPU:

- Executes parallel code

So the program is divided into:

- Host code (runs on CPU)
- Device code (runs on GPU)

CUDA built-in variables: Inside kernel, CUDA gives built-in variables:

Variable	Meaning
threadIdx.x	Thread ID inside block
blockIdx.x	Block ID inside grid
blockDim.x	Number of threads in block
gridDim.x	Number of blocks in grid

Thread indexing:

myKernel<<<3, 4>>>();

That means:

- 3 blocks
- 4 threads per block
- Total = 12 threads

To calculate global thread ID:

int id = blockIdx.x * blockDim.x + threadIdx.x;

This gives unique ID for each thread.

Example:

Block 0:

- threadIdx: 0,1,2,3
- id: 0,1,2,3

Block 1:

- threadIdx: 0,1,2,3
- id: 4,5,6,7

Block 2:

- threadIdx: 0,1,2,3
- id: 8,9,10,11

Now each thread works on different data.

Memory transfer: CPU and GPU have separate memory.

So we must:

- Allocate memory on GPU
- Copy data from CPU to GPU
- Run kernel
- Copy result from GPU to CPU

Basic CUDA functions:
cudaMalloc(): Allocates GPU memory

cudaMemcpy(): Copies data from CPU to GPU

cudaFree(): Free GPU memory

1D, 2D and 3D grids: In CUDA, threads and blocks can be arranged in 1D, 2D, or 3D grids, depending on the structure of the data being processed.

CUDA supports:

- 1D threads (vectors)
- 2D threads (images, matrices)
- 3D threads (volumetric data)

A **1D thread grid** is used when the data is arranged in a single linear dimension. Each thread processes one element of the vector or list. Example: Vector addition

Thread indexing:

$$id = blockIdx.x \times blockDim.x + threadIdx.x$$

This gives a unique thread ID. Each CUDA thread computes one vector element.

CUDA code example:

```
__global__ void vectorAdd(int *A, int *B, int *C, int n)
{
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   if(i < n)
      C[i] = A[i] + B[i];
}
```

Kernel launch configuration:
vectorAdd<<<(n+255)/256 , 256>>>(A,B,C,n);

Here, the number of blocks is determined by the vector size and 256 threads per block.

A **2D thread grid** is used when data is arranged in rows and columns, like images, matrices and grids. Example: Image filtering, edge detection, matrix multiplication, computer vision.

Thread indexing:

x = blockIdx.x * blockDim.x + threadIdx.x

y = blockIdx.y * blockDim.y + threadIdx.y

Each thread processes one pixel or matrix element.

CUDA code example:

```
__global__ void brighten(unsigned char *image, int width, int height)
{
   int x = blockIdx.x * blockDim.x + threadIdx.x;
   int y = blockIdx.y * blockDim.y + threadIdx.y;
   int index = y * width + x;
   if(x < width && y < height)
      image[index] += 10;
}
```

Kernel launch configuration:

```
dim3 block(16,16);
dim3 grid(width/16, height/16);
brighten<<<grid,block>>>(image,width,height);
```

A **3D thread grid** is used when the dataset has three spatial dimensions. Example: CT scan volume, MRI data

Thread indexing:

x = blockIdx.x * blockDim.x + threadIdx.x

y = blockIdx.y * blockDim.y + threadIdx.y

z = blockIdx.z * blockDim.z + threadIdx.z

Each thread processes one **voxel**. A voxel is a 3D pixel.

A CT scan may produce: 512 × 512 × 200
This means 512 pixels width, 512 pixels height and 200 slices (depth)

Total voxels: 512 × 512 × 200 = 52,428,800 voxels

Each voxel stores tissue density. A CUDA thread can process one voxel.

CUDA code example:

```
__global__ void processVolume(float *volume, int width, int height, int depth)
{
   int x = blockIdx.x * blockDim.x + threadIdx.x;
   int y = blockIdx.y * blockDim.y + threadIdx.y;
   int z = blockIdx.z * blockDim.z + threadIdx.z;

   int index = z*width*height + y*width + x;
   if(x < width && y < height && z < depth)
      volume[index] *= 2;
}
```

Kernel launch configuration:

```
dim3 block(8,8,8);
dim3 grid(width/8, height/8, depth/8);
processVolume<<<grid,block>>>(volume,width,height,depth);
```

Purpose of cudaDeviceSynchronize():

In CUDA programming, GPU operations (especially kernel executions) run asynchronously with respect to the CPU. This means the CPU continues executing the next instructions without waiting for the GPU to finish its work.

The function cudaDeviceSynchronize() is used to pause the CPU until all previously launched GPU tasks are completed.